

Reviewer Pack — Minimal Local Ring Norm Correlation with Resolution Proof Wi...

Entry 5ae0a888f254 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 11:23:50 UTC

Minimal Local Ring Norm Correlation with Resolution Proof Width via LLL Reduction

Entry ID: 5ae0a888f254 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 11:23:50 UTC

1. Conjecture

Field A (mathematical branch): Local Ring Theory **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every Boolean formula φ , the minimal local ring norm ($\text{minLRN}(\varphi)$) of its associated monomial representation is linearly correlated with its resolution proof width $w(\varphi)$, such that $\text{minLRN}(\varphi) = \Theta(w(\varphi))$.

Rationale (proposer's reasoning):

Local rings provide a natural setting to study arithmetic properties of Boolean functions. By leveraging LLL reduction, we can transform the resolution proof process into an arithmetic problem within these local rings. This connection could potentially reveal new insights into the complexity of resolving Boolean formulas.

Taxonomy category: local_ring_theory_resolution (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8c3d581755cded77

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, across all tested Boolean formulas φ with n variables ($n \leq 40$), the Pearson correlation coefficient r is greater than or equal to 0.8 AND the mean of $\text{minLRN}(\varphi)$ divided by $w(\varphi)$ falls within the range $[1, 10]$. The conjecture is falsified if either condition fails.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal local ring norm" AND "resolution proof width" - "monomial representation" AND LLL reduction IN local ring theory AND resolution complexity- "0-linear correlation" BETWEEN minLRN AND $w(\phi)$ IN field_A AND field_B

Top relevant hits considered: - [http://arxiv.org/abs/2409.14352v2] Centraliser algebras of monomial representations and applications in combinatorics - [http://arxiv.org/abs/2410.03444v2] Factoring through monomial representations: arithmetic characterizations and ambiguity of weighted automata - [http://arxiv.org/abs/2406.14513v3] On fibered Burnside rings, fiber change maps and cyclic fiber groups - [http://arxiv.org/abs/math-ph/9805020v2] Chern-Simons Terms as an Example of the Relations Between Mathematics and Physics - [http://arxiv.org/abs/1002.1146v1] Interconnection between 802.15.4 Devices and IPv6: Implications and Existing Approaches - [http://arxiv.org/abs/1002.1157v1] Establishment of Relationships between Material Design and Product Design Domains by Hybrid FEM-ANN Technique - [s2:10.1093/europace/eut316] characterization of left atrial wavefront activation during human persistent af using a 3D non-contact mapping High accu - [s2:2b3cdd1442e58a5bd6096d68f67f129cd64939c0] Automata, languages and programming : 28th International Colloquium, ICALP 2001, Crete, Greece, July 8-12, 2001 : procee

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
    for i in range(m):
        max_row = i
        for j in range(i + 1, m):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        pivot = A[i][i]
        for j in range(n):
            A[i][j] /= pivot
        for j in range(m):
            if j != i:
                factor = A[j][i]
                for k in range(n):
                    A[j][k] -= factor * A[i][k]
```

```

def matrix_multiplication(A, B):
    m, n, p = len(A), len(B), len(B[0])
    C = [[0] * p for _ in range(m)]
    for i in range(m):
        for j in range(p):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def min_local_ring_norm(formula):
    # Placeholder implementation
    return random.random()

def resolution_proof_width(formula):
    # Placeholder implementation
    return random.randint(1, 10)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    instances_tested = 0
    min_lrn_sum = 0
    w_sum = 0
    counterexample = ""

    for _ in range(30):
        formula = '& '.join(random.choices(['x1', 'x2', 'x3'], k=n))
        min_lrn = min_local_ring_norm(formula)
        w = resolution_proof_width(formula)

        if math.isnan(min_lrn) or math.isinf(w):
            continue

        instances_tested += 1
        min_lrn_sum += min_lrn
        w_sum += w

        if instances_tested >= 30:
            break

    if instances_tested < 30:
        return {
            "metric_name": "minLRN vs. Resolution Width",
            "metric_value": None,
            "instances_tested": instances_tested,
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": "insufficient_instances"
        }

    mean_min_lrn = min_lrn_sum / instances_tested
    mean_w = w_sum / instances_tested
    correlation_coefficient = (instances_tested * sum(min_lrn * w for min_lrn, w in zip(min_lrn_list, w_list)) -
                             mean_min_lrn * mean_w) / math.sqrt((instances_tested * sum(min_lrn**2 for min_lrn in min_lrn_list) -
                             (mean_min_lrn**2) * instances_tested) * (instances_tested * sum(w**2 for w in w_list) -
                             (mean_w**2) * instances_tested))

```

```

if correlation_coefficient >= 0.8 and 1 <= mean_min_lrn / mean_w <= 10:
    return {
        "metric_name": "minLRN vs. Resolution Width",
        "metric_value": correlation_coefficient,
        "instances_tested": instances_tested,
        "n_max": n,
        "conjecture_holds": True,
        "counterexample": ""
    }
else:
    return {
        "metric_name": "minLRN vs. Resolution Width",
        "metric_value": correlation_coefficient,
        "instances_tested": instances_tested,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": f"correlation={correlation_coefficient}, mean_min_lrn/mean_w={mean_min_lrn/mean_w}"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    mean_metric_value = sum(r['metric_value'] for r in results if r['metric_value'] is not None) / len(results)
    std_metric_value = math.sqrt(sum((r['metric_value'] - mean_metric_value)**2 for r in results if r['metric_value'] is not None) / len(results))
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

    if all(r['conjecture_holds'] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif sum(1 for r in results if not r['conjecture_holds']) / len(results) >= 0.2:
        print(f"RESULT: FALSIFIED counterexample=\"{results[next(i for i, r in enumerate(results) if not r['conjecture_holds'])]}\"")
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_96dc6b24.py", line 112, in <module>

result = run_trial(seed)

^^^^^^^^^^^^^^^^

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_96dc6b24.py", line 93, in run_trial

min_lrn = min_local_ring_norm(formula)

^^

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_96dc6b24.py", line 38, in min_local_rin
    M = monomial_representation(formula)
    ~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_96dc6b24.py", line 34, in monomial_repr
    left, operator, right = formula.split('&', 1)
    ~~~~~
```

```
ValueError: not enough values to unpack (expected 3, got 2)
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that it was unable to complete the computation necessary to evaluate the conjecture's support conditions. | next: Re-run the test with appropriate error handling and debugging to ensure it completes successfully.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13203
2	preregistration	ollama_remote	glm4:latest	0	0	13144
3	novelty	ollama_remote	glm4:latest	0	0	13276
4	novelty	ollama_remote	glm4:latest	0	0	26055
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25797
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16976
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22887
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16362
9	judge	ollama_remote	glm4:latest	0	0	8606

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 156306 ms total latency. Provider mix: {'ollama remote': 9}

(full prompt+response transcripts available in [research/audit/5ae0a888f254.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5ae0a888f254.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/5ae0a888f254.tar.gz (if generated)